

EMUQU — Enforced Scientific Honesty: Build-Time Governance of a Consumer Physiology Score

Constants-hash version locks, a machine-checked science register, gates that must prove they fail, and one-way debt ratchets

Technical White Paper • v1.0

October 2026

Chris Sharp

Describes Emuqu at source revision e028039 (2026-10-07); scoring version v3.1.oct2026.

Executive Summary

Emuqu is an iOS app that turns overnight beat-to-beat (RR) intervals from a chest strap, with sleep and vitals data, into a daily recovery score. Like every consumer composite score, its weights and bands are hand-calibrated: some inputs rest on published methods, others are product rules with no outcome evidence. The app is not a medical device. It still has to stop four kinds of quiet drift:

1. A scoring constant changes, and stored history becomes a silent mixture of two algorithms under one name.
2. A heuristic's evidence status is forgotten, and user-facing copy starts to imply more than the evidence supports.
3. A quality gate reports "clean" while measuring nothing.
4. A debt ceiling is raised to make the build pass instead of paying the debt.

This paper describes the build-time machinery the repository uses against each of these. All of it is plain bash and Python run in continuous integration (CI):

- **A constants-hash version lock.** The numeric constant declarations in two designated scoring files (155 at this revision) are reduced to a sorted `file:symbol=value` list, hashed with SHA-256 and truncated to 16 hex characters. The build fails if that digest moves while the scoring-version string still equals the recorded version. Every stored score breakdown carries its version, and records from before versioning decode to an explicit `unversioned` sentinel.
- **A machine-checked science register.** A JSON inventory classifies 19 heuristics into a closed status vocabulary (3 `validated`, 4 `supported-transfer`, 12 `awaiting-validation`, 0 `scheduled-for-removal`). A gate checks fields, cited test paths, version agreement with the code and retracted phrasings, and requires every score-adjusting function found by a name scan to be claimed or waived.

- **Gates that test the gates.** Each cheap gate must go red on a planted violation (64 plants), fail closed on temp-file errors, and be wired into CI. A separate harness applies 125 targeted production mutations that named test suites must catch, and a static backstop detects any mutation left in the tree.
- **One-way ratchets.** Each CI budget file may move only in its tightening direction, read from the filename. A loosening needs a `budget - raise - ok`: commit trailer, and docs may not restate a ratcheting value.

The paper also states where these mechanisms stop. On 2026-10-06, for example, the constants hash was re-recorded without a version bump and the gate passed, because it compares only against the recorded line.

1. The Problem

1.1 Composite scores are calibrated, not validated

Consumer wearables publish composite "readiness" or "recovery" numbers built from heart-rate variability (HRV), sleep and other signals. The component measurements have a literature: the Task Force standards for time-domain HRV [1], and the smallest-worthwhile-change method on a rolling $\ln(\text{RMSSD})$ baseline [2]. The way they are combined usually does not. Reviews of consumer wearables in athletic populations note that the proprietary algorithms are rarely validated independently [3]. Some popular sub-heuristics have been challenged directly. The LF/HF ratio as an index of "sympathovagal balance" is one [4]. The acute:chronic workload ratio as an injury-risk signal is another [5]. Emuqu's own register records that its composite weights are calibrated, not outcome-validated.

A product in this position can be honest in two ways. It can remove every unvalidated rule, which in practice removes the product. Or it can keep bounded heuristics and make their status impossible to forget. Emuqu takes the second route, and the second route only works if forgetting is mechanically prevented.

1.2 Sources of drift

Drift	How it happens	Why review alone misses it
Algorithm mixing	A threshold or weight is retuned; stored scores computed before and after look identical	The diff touches one number, and nothing marks history
Claim creep	Copy is reworded ("strong predictor" becomes "strongly suggests")	A lexical banned-word list matches phrasings, not claims
Dead evidence	A test cited as backing for a rule is deleted or renamed	Nobody re-reads the evidence inventory
Resurrected errors	A figure corrected in one file survives in another	Corrections are local; copies are not
Fail-open gates	A temp file cannot be created; the gate greps an empty file and prints "clean"	A green tick looks the same whether or not anything was measured
Unwired gates	A gate exists in the scripts directory but no CI step runs it	The directory listing looks complete
Goalpost moves	A budget file is edited upward so CI passes	The edit is one character

The repository's own comments record that each row happened at least once. A gate once reported clean after `mktemp` failed. A planted test mutation (a pNN50 threshold of 75 ms instead of 50 ms) was left in the tree by a killed run and committed. Two gates were wired into the Makefile only. A retracted reliability figure survived in an analyzer header after the register had corrected it.

1.3 Why existing approaches are incomplete

- **Lint baselines** (SwiftLint's `--baseline`, betterer [6]) freeze a snapshot of existing violations so new ones fail. They do not say which direction a numeric limit may move, they do not cover coverage floors, and they do not stop a maintainer regenerating the baseline.
- **Model cards and model registries** [7][8] attach documentation and version identifiers to machine-learning models. They describe a model. They do not fail a build when a hand-set constant changes without a version change, and they do not tie a claim inventory to the test files that back it.
- **Mutation testing frameworks** (PIT [9], Stryker [10]) mutate broadly and report a mutation score against a threshold. A percentage does not say which high-stakes behaviour went unasserted, and a full run on a mobile codebase with a simulator-bound test suite is expensive.
- **Medical-device quality systems** (IEC 62304 [11], ISO 13485 [12]) require traceability from requirements to verification and controlled change. They are process standards, audited by people. They do not prescribe build-time mechanisms of the kind described here, and Emuqu is not developed under them.

What is missing is a small set of executable checks that bind three things together: the numbers in the scoring code, the version string on stored scores, and the evidence status and permitted wording of each heuristic. A further set of checks is needed to prove that those checks themselves work.

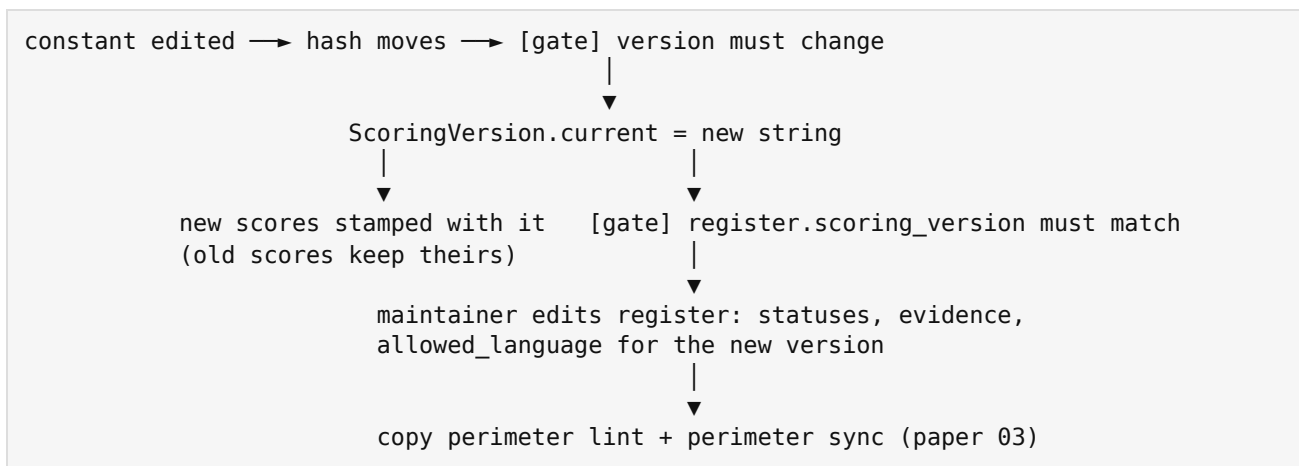
2. Architecture Overview

The governance layer is a set of scripts in the repository, run in two CI workflows and by a local `make ci` target. None of it runs on the user's device. The on-device parts are the version stamp on stored scores and the text that shows it.

Phase	Operation	Purpose
Author time	Change a scoring constant	—
Gate: scoring governance	Hash numeric constants; compare with recorded <code>version hash line</code>	Force a version bump when the numbers move
Gate: scoring governance	Scan app sources for the ranking-weight identifier outside one module	Keep a hand-set ordering key from being displayed as a confidence
Runtime	Stamp every new score breakdown with the current version; decode unstamped history as <code>unversioned</code>	Keep each stored score attributable to its algorithm
Gate: science register	Register version must equal code version	Force an edit of the register on every version bump
Gate: science register	Status vocabulary, required fields, test paths, retracted claims, discovery scan, entry floor	Keep the claim inventory complete in a checkable sense and keep it current

Phase	Operation	Purpose
Gate: copy perimeter (companion paper 03)	Prohibited-term lint on Swift string literals and string catalogues; build-time list reconciled with the runtime assistant guards	Keep medical-claim vocabulary out of user-facing text
Meta-gates	Planted violation per gate; fail-closed temp files; wiring check	Prove each gate goes red, fails closed and is reached
Test verification	Targeted mutation per behaviour; leak backstop; tree lock	Prove the test suites assert the behaviours that matter
Ratchets	Direction-aware budget comparison against a baseline commit	Debt only falls, floors only rise
Reporting	Evidence report prints live counts	Give reviewers numbers that cannot be stale

The chain that matters most for scientific honesty, shown as a sequence:



3. Methods in Detail

3.1 Scoring-constants hash and version lock

Inputs.

- Two designated scoring-constant source files: the recovery-score constants file (91 numeric declarations) and the HRV thresholds file (64 numeric declarations). That makes 155 at this revision.
- The scoring-version source file, which declares `current` and `unversioned`.
- A one-line baseline file in the CI directory containing `<recorded_version> <recorded_hash>`. At this revision it reads `v3.1.oct2026 541b56e600080795`.

Steps.

1. Visit the two constant files in a fixed order: recovery-score constants first, HRV thresholds second. If either file is missing, exit 2 (configuration error).

2. Read each file and delete every `//` line comment. The regex is `//[^\n]*`, replaced with nothing. Rewording a rationale comment therefore never changes the hash. Block comments are not stripped; neither file contains one.
3. Extract all matches of `static let\s+(\w+)\s*:\s*?\s*[\w<>\[\], .]*=\s*([-\.d.]+)` giving (symbol, value) pairs. value is the literal text of the number as written, so `60` and `60.0` are different values. A declaration whose right-hand side is not a bare numeric literal is not matched.
4. Sort the pairs of each file lexicographically by (symbol, value). Moving a declaration within a file, or renaming an enclosing namespace, does not change the hash. Renaming a symbol or changing its value does. Symbols repeated under different nested namespaces are all included.
5. For each pair, in file order and then sorted order, feed the UTF-8 bytes of `"<file path>:<symbol>=<value>\n"` into one running SHA-256 digest.
6. Take the first 16 hexadecimal characters of the digest (64 bits) as `current_hash`.
7. Read `current` from the version file with `static let current\s*=\s*"([^\"]+)"`. If it cannot be read, exit 2.
8. If the baseline file is missing, exit 2. Otherwise split it on whitespace into `recorded_version` and `recorded_hash`.
9. **Violation rule:** if `current_hash != recorded_hash` and `version == recorded_version`, report "scoring constants changed while the version stayed the same" and exit 1.
10. Print the number of constants hashed, the hash and the version, whatever the result.

Ranking-weight confinement (same script). The probable-cause list orders candidate causes by a hand-set number named `rankingWeight`. Its register entry records that there is no outcome evidence behind it. The gate walks every Swift file under the app sources. Any line containing the substring `rankingWeight` in a file outside the cause-detection directory is a violation. The identifier therefore cannot reach a view, an export, analytics or accessibility text unless it is renamed. A rename is a visible change, and the register entry states that the value must never be rendered as a confidence or probability.

Version stamp at runtime.

Rule	Behaviour
Construction	A new <code>ScoreBreakdown</code> defaults its <code>scoringVersion</code> field to the current version
Decoding	A stored breakdown with no <code>scoringVersion</code> key decodes to the literal string <code>unversioned</code> , never to the current version. Scores from before the stamp may come from v1 or v2, because the app offered a history recompute the user may or may not have run.
Derived copies	Code that rebuilds a breakdown copies the version field forward. This covers the vitals refresh after scoring and the stripped copy prepared for iCloud sync.
Display	The recovery-score detail screen shows "Scored by v3.1.oct2026" (the stored value), or "Scored before score versions were recorded" for <code>unversioned</code> . Beneath it is a fixed line saying the inputs are research-informed and the weights and bands are calibrated, not outcome-validated.
Assistant	The in-app assistant's fact catalogue (key <code>score.algorithm.version</code>) and its context block report the <i>current</i> version string and describe the v1 weighting older history may carry. They do not report the per-score stamp.

The version is not printed on every surface. At this revision only the score-detail screen renders the stored stamp. Dashboard rings, widgets and reports do not.

Version history in the public repository.

Version	Introduced	Recorded hash
v2.may2026	2026-09-08 (public initial commit)	40848e4011cfc32b
v3.oct2026	2026-10-03	c6c8520fd1d48454
v3.1.oct2026	2026-10-04	b81e8c3b2a364000
v3.1.oct2026 (re-recorded)	2026-10-06	541b56e600080795

The last row is the limitation discussed in Section 5.1. Commit 14ecd58 replaced two monotony-strain thresholds in the training-readiness constants with a single one, and it re-recorded the hash under the same version string. The gate passed.

3.2 The science register

Data model. One JSON file with these top-level keys:

Key	Content
<code>_purpose</code>	Prose: a governed inventory of heuristics that change a number the user sees
<code>_what_the_gate_can_and_cannot_prove</code>	Prose statement of the gate's limits (quoted in Section 5)
<code>_retracted_claims.claims</code>	List of evidence phrasings that were once stated and turned out wrong (3 strings at this revision: two spellings of one reliability range, and one citation to a replication study)
<code>_status_meanings</code>	Map from each allowed status word to its definition. The gate reads the vocabulary from this map.
<code>_waived_symbols</code>	Map from function name to the reason it is not an entry (1 at this revision: a display-only translation helper)
<code>scoring_version</code>	Must equal the code's current version
<code>entries</code>	List of classified heuristics

Status vocabulary (exact words and meanings from the register).

Status	Meaning (register text, abridged)	Count
validated	An external peer-reviewed result supports this specific use, or the repository verifies it against reference data on every build	3
supported-transfer	A real published result exists, but the app applies it outside the population, protocol or endpoint it was established in. Defensible, not established.	4
awaiting-validation	A hand-calibrated product rule with no outcome evidence for these particular numbers. Retained because it is bounded and removing it would change stored scores; version-stamped so a later removal is visible.	12

Status	Meaning (register text, abridged)	Count
scheduled-for-removal	Kept only until a replacement lands	0

Entry fields.

Field	Required by the gate	Role
id	Unique	Stable name
status	Must be a key of <code>_status_meanings</code>	Evidence level
affects	Non-empty	Which user-visible outputs change
evidence	Non-empty	What supports or does not support the rule, with sources
allowed_language	Non-empty	The wording copy may use, and the wording it must never use (for example "Damper, bookkeeping. Never injury risk or dangerous.")
tests	Optional; every path listed must exist	Tests that pin the behaviour
symbols	Optional	Function names in the analysis layer that this entry claims

Gate steps.

- Exit 2 if the register, the entry-floor file or the version source is missing, if the JSON does not parse, or if `_status_meanings` is empty.
- For each entry, record a problem for a duplicate `id`, a status outside the vocabulary, an empty `affects`, `evidence` or `allowed_language`, or a `tests` path that does not exist in the working tree.
- Read `current` from the version source and record a problem if the register's `scoring_version` differs. Every version bump therefore fails this gate until the register is edited.
- Retracted claims.** For each retracted string, search for an exact substring match in:
 - each entry's `evidence`, `affects`, `allowed_language` and `notes` fields (lists joined with spaces);
 - every `.swift` and `.md` file under the app sources and the docs directory.

The retracted list itself is the only place the strings may appear. This paper describes them without quoting them for that reason.

- Discovery.** Over every Swift file in the analysis directory, excluding the cause-detection subdirectory, collect the names matched by `func\s+([a-zA-Z]+(?:Adjustment|Penalty|Bonus|Modifier|Damper|Fraction))\b`. Discard names ending in `Descriptions` or `Message`. At this revision the scan finds 13 functions: 12 claimed and 1 waived.
- Let `claimed` be the union of all entries' `symbols`, and `waived` the non-underscore keys of `_waived_symbols`.

- Each discovered name in neither set is a problem ("a score-adjusting function no register entry claims").
- Each name in `claimed ∪ waived` that was not discovered is a problem ("claimed or waived but no longer exists"). A stale claim fails, just as a missing one does.

7. Print a tally by status, plus the discovered, claimed and waived counts. If there are any problems, exit 1.

8. **Entry floor.** Read the minimum entry count from the CI directory's register floor file. If the number of entries is below it, exit 1 with instructions: a heuristic still in the code must stay classified, and a heuristic removed from the code needs the floor lowered through the audited trailer of Section 3.5. If the count is above the floor, print a reminder to raise it.

The floor file's name begins with `min_`, so the ratchet of Section 3.5 treats it as a floor that may only rise.

3.3 The copy perimeter, briefly

Companion paper 03 covers the copy perimeter in detail. In outline, a Python linter reads a JSON list of prohibited regular expressions and scans every Swift string literal and string-catalogue value for user-facing medical-claim vocabulary. Comments are stripped, and an allowlist covers the methodology page and the guard sources. A second gate proves that the runtime vocabulary used by the in-app assistant's input and output guards covers every build-time pattern. For each build-time regex it synthesizes a matching string and requires some runtime lexicon concept to match it too.

For this paper, the point is the division of labour. The perimeter is lexical: it can ban a phrasing but not a claim. The register's `allowed_language` field states the claim boundary per heuristic. No gate checks copy against `allowed_language`. That link is made by the maintainer, who is forced to revisit the register on every version bump (step 3 of 3.2).

3.4 Gates that test the gates

The repository has 45 gate scripts (names beginning `check_` or `enforce_`). Three meta-checks establish three properties of every gate: it **goes red** on a real violation, it **fails closed** when it cannot measure, and it **is wired** into CI.

3.4.1 Goes red: planted violations

For each plant, defined as `(name, gate command, file to mutate, mutation program)`:

1. If a name filter was given and does not match, skip.
2. Run the gate. If it is already red, print SKIP ("fix that first") and continue. A red-after-mutation result proves nothing if the gate was red before.
3. Copy the target file to a backup directory, which was created through the fail-closed helper of 3.4.2.
4. Apply the mutation, a short Python program that edits the file. Most mutations assert that the original text occurs the expected number of times. If the mutation fails, print SKIP and restore.
5. Run the gate again, record its exit code, and restore every touched file.
6. A non-zero exit is PASS. A zero exit is FAIL ("STAYED GREEN — this gate protects nothing").

At this revision there are 64 plants. Examples: a retracted phrase appended to a register evidence field; an unknown register status; the hash baseline zeroed under the same version; a budget raised by 100000 with no trailer; a CI step renamed to a nonexistent script; the pNN50 mutation left in place; a bare `mktemp` in a gate. Several gates have one plant per shape of miss they have exhibited.

Meta-check: every gate has a plant. After the plants run, the harness lists every `check_*.sh` and `enforce_*.sh` in the scripts directory and drops two groups. The first is a two-entry `NEEDS_BUILD` exemption list (the coverage-floor gate and the Thread Sanitizer gate, which need a build or a simulator). The second is the preflight self-check, whose name is excluded from the meta-check although it does have two plants. For each remaining gate it greps the harness's own source for the gate's filename. Any gate whose filename does not appear makes the run exit 1, listing the unplanted gates. Adding a gate and adding its plant must therefore happen in the same change.

The run also exits 1 if any plant was FAIL. A plant that was SKIPped does not fail this harness, which differs from the test-mutation harness in 3.4.4.

3.4.2 Fails closed: temp-file preflight

The gates run with `set -uo pipefail` but not `-e`, because several rely on `grep` returning non-zero when nothing matches. Without `-e`, a failed `mktemp` leaves an empty path, and later reads of that path find nothing, which the gate reports as "clean". The fix has two halves.

Helper library.

- `gate_mktemp label` creates `${TMPDIR:-/tmp}/label.XXXXXX` with an explicit template, which behaves the same under BSD and GNU `mktemp`. If creation fails, the path is empty, or the file is not writable, it prints the reason and calls `exit 70 (EX_OSERR)`.
- `gate_mktemp_dir` does the same for directories.
- `gate_require_nonempty` exits 70 when an input the gate is about to parse is missing or empty.

Each helper runs inside `$( )`, which is a subshell, so its `exit` cannot stop the caller. Every call site must therefore append `|| exit $?`.

Preflight gate. For every `.sh` file in the scripts directory except itself, it scans each non-comment line and reports:

1. any `mktemp` not preceded by `gate_` on a line that does not contain `gate_mktemp`;
2. any line calling `gate_mktemp` without `|| exit` on the same line;
3. any `source "${dirname "$0"}/<path>"` whose target does not exist, because a mistyped source path would silently define no helpers.

Any finding exits 1. At this revision it scans 53 scripts and finds 10 guarded temp-file uses.

The ratchet comparison (3.5) and the scoring and register gates also fail closed on configuration problems: a missing input exits 2, not 0.

3.4.3 Is wired: CI reachability

The wiring gate reads the manual full-CI workflow file. The path can be overridden by an environment variable; a missing file exits 2. Every `check_*.sh` or `enforce_*.sh` whose filename does not occur as a substring of that file is reported. One gate is exempt by name with a recorded reason: the coverage-floor gate, which runs inside the coverage step. Any unwired gate exits 1. At this revision 44 gates appear in the workflow and 1 is invoked by another step.

3.4.4 The tests must catch defects: targeted mutations

This harness applies the planted-violation discipline to the unit tests instead of the gates. It deliberately does not mutate broadly. It holds a curated list of 125 mutations, each defined as `(name, test suite identifier, production file, mutation program)`. Examples: widen the pNN50 threshold from 50 ms to 75 ms; return the mean instead of the standard deviation for SDNN; feed raw RR intervals instead of successive differences to RMSSD; shift the top verdict band from 90–100 to 91–100; remove the finiteness guard on a date-to-milliseconds conversion; drop the detrend step from DFA.

For each mutation:

1. Run the named suite alone (`-only-testing`, parallel testing off) and classify the result:
 - **0** — passed;
 - **1** — tests failed;
 - **2** — build or runner failure. This is recognised by a build-failure banner in the log, or by the absence of any "Test Suite" or "Test Case" line.
2. If the baseline result is 2, retry once. If the baseline is still not 0, print SKIP with the reason.
3. Back up the file and apply the mutation. Each mutation asserts that its original text occurs exactly once, so a mutation cannot silently land in a doc comment or the wrong place. If it cannot be applied, print SKIP.
4. Run the suite and restore.
5. A non-zero result is PASS ("caught"). A zero result is FAIL ("SURVIVED — asserts nothing about this").
6. At the end, **any FAIL or any SKIP exits 1**. "A skip is not a pass."

This harness needs a simulator and a build per mutation. It is not part of `make ci`, and no CI workflow runs it; it is run by hand when analysis or storage code changes.

3.4.5 Backstops: leaked-mutation detection and the tree lock

Both harnesses edit tracked files in place and restore them in an EXIT trap. A killed process skips the trap.

Leak detector. A static gate parses the mutation harness's source:

1. Split the source on the literal line prefix `mutate "`.
2. For each case, take the target path: the first double-quoted string that starts with the app directory and ends in `.swift`. Take the original text from either `o='...';assert` or `s.count('...')` followed by a comparison operator.
3. Unescape `\'`, `\"`, `\n` and `\$`. Resolve `' + chr(N) + '` concatenations to the character `N`.
4. If the original still contains `chr(` or a string concatenation, mark the case unparsed.

5. If the target file is missing, or the original text is not present in it, report a leak.
6. Any leak exits 1. **Any unparsed case also exits 1**, because a case the parser cannot read is a case it is not checking.

At this revision all 125 cases parse and none has leaked. This gate runs on every push.

Tree lock. Before mutating, each harness atomically creates a lock directory inside the git directory using `mkdir`, and writes its name and PID into it. A second mutator finds the lock; if the PID is alive, it exits 75 (`EX_TEMPFAIL`). If the PID is dead, it warns that the earlier run never restored, points to the leak detector, and takes the lock over. Read-only checks can call a reader function that refuses to run (exit 75) while a live mutator holds the lock. The lock is released in the same trap that restores files.

3.5 One-way ratchets

Decision record. The repository's architecture decision record on ratcheted budgets (ADR 004) states the policy. Every quality rule has a measured numeric ceiling, never an aspirational one. A ceiling may only move down. There is no per-site escape hatch: the app sources contain no SwiftLint disable directives. The record reasons that a gate nobody can satisfy gets routed around, while a measured ceiling blocks regression immediately.

Counting code, not prose. Several budgets count occurrences of a pattern: static singleton reads, `try?`, legacy observable-object conformances. Before counting, the counter removes comments. Block comments `/* ... */` are blanked character for character, keeping newlines. Line comments `//...` are replaced with spaces of the same length, so nothing on an adjacent line is joined into a match. Explaining a piece of debt in a comment therefore never raises its count. A fixed list of platform singletons (`URLSession`, `FileManager`, `NotificationCenter` and others) is blanked before the singleton count, because no version of the app can inject those away.

Monotonicity gate.

1. **Baseline resolution**, in order:
 - an explicit argument;
 - the pull-request target branch (`origin/<GITHUB_BASE_REF>`);
 - a `BUDGET_BASE_REF` environment variable;
 - otherwise there is no baseline.
2. **No baseline.** If `BUDGET_REQUIRE_BASELINE` is set, exit 1 ("refusing to report enforcement that did not happen"). Otherwise print a skip notice and exit 0. The same applies to an all-zero SHA (a new or force-pushed branch) and to a ref that does not resolve.
3. **Acknowledgements.** Collect them from every commit message in `baseline..HEAD`, using the case-insensitive regex `^budget-raise-ok:[[:space:]]*[^[:space:]]+`. Keep only the path token.
4. **Comparison.** For each `*.txt` file directly in the CI directory, in sorted order:
 - Read the current value. It must match `^[0-9]+$` once whitespace is removed; otherwise skip the file.
 - Read the value at the baseline with `git show`. A file absent at the baseline is a new budget and is skipped. A non-integer baseline value is skipped too.
 - Equal values pass.
 - **Direction** comes from the filename. A basename starting with `min_` is a floor, and a decrease loosens it. Any other file is a ceiling, and an increase loosens it.

- A loosening whose exact path appears among the acknowledged paths prints `ALLOWED`. Any other loosening is a violation. A tightening prints `ratcheted`.

5. Any violation exits 1, with instructions to pay the debt or add `budget-raise-ok: <path> <why>`.

CI wiring of the baseline. The manual workflow resolves the baseline to the parent commit, or to the working tree for a root commit with budget edits, and sets the `require` flag whenever a baseline exists. An unresolvable baseline is a hard failure. The per-push workflow passes the pull-request target, or the parent commit with `BUDGET_REQUIRE_BASELINE=1` on a direct push to main. A push to main therefore cannot skip the comparison.

Documentation may not restate a ratcheting value. The documentation-links gate scans every Markdown file. Whenever a line names a CI budget file whose content is a plain integer, it looks for a bare 2–6-digit number in that line or the next one, using the regex `(?![\w. ])(\d{2,6})(?![\w.%])`. Any such number is reported. The one exception is when the window contains a backtick code span with `cat`, `head` or `read_budget`, which is a live read and cannot go stale. Docs must point at the file instead of quoting it. This paper follows that rule, and gives no ratcheting budget value next to its file name.

Observed history. Across the 28 commits of the public repository (2026-09-08 to 2026-10-07), every integer budget that changed moved in its tightening direction. The SwiftLint warning, `try?`, test-skip and localization-orphan ceilings each fell more than once, and the refactor-spec watch ceiling fell to zero. The views-coverage floor rose. No commit message carries a `budget-raise-ok: trailer`.

3.6 Costed CI posture and the evidence report

macOS CI minutes bill at ten times the Linux rate. The repository measured a full macOS run at about 108 wall-clock minutes, roughly 1,080 billable, against a plan of about 300 macOS minutes a month. The posture follows from that arithmetic:

Workflow	Trigger	Runner	Content
Script gates	Every push to main and every pull request	Linux	Every gate that needs only bash, Python and git: copy perimeter, perimeter sync, science register, scoring governance, localization, source-hygiene guards (including the leaked-mutation backstop), documentation and SBOM checks, wiring and preflight meta-gates, tech-debt budgets, and the monotonicity ratchet
Full CI	Manual, or called by the release workflow	macOS	Scope <code>gates</code> (about 14 billable minutes): SwiftLint and its budget, all budgets, the planted-violation harness and every gate step. Scope <code>unit</code> adds unit tests and the Thread Sanitizer. Scope <code>full</code> adds UI tests and coverage floors.
Local	<code>make ci</code>	Developer machine	Every gate, the planted-violation harness and the full test suite with coverage
Manual	By hand	Developer machine with simulator	Targeted mutation harness (3.4.4)

The planted-violation harness is not in the per-push Linux workflow, because SwiftLint behaves differently on Linux. It runs in the manual workflow and in `make ci`.

The **evidence report** script prints, in seconds and without building, live counts (files, tests, locales, waivers) and every budget value, as text or JSON. It omits coverage and pass/fail results on purpose: those need a simulator, and a cached number would be stale.

3.7 How the parts form one chain

1. A developer edits a numeric threshold in a scoring-constant file. **Scoring governance** fails: the digest moved and the version did not.
2. The developer changes `ScoringVersion.current` and re-records the baseline. New breakdowns carry the new stamp; stored ones keep theirs, and the score-detail screen shows each record's own.
3. **Science register** now fails until its `scoring_version` is edited. That is the moment to revise each affected entry's status, evidence and `allowed_language`. The gate checks that the file was touched, not that the revision was made.
4. A new function with a score-adjusting name fails discovery until claimed or waived. A removed one fails the stale-claim check. A removed heuristic fails the entry floor unless the floor is lowered with an audited trailer.
5. New user-facing copy passes through the **copy perimeter** (paper 03).
6. Each of these gates is proven to go red, is reached by CI and fails closed, and every budget the change touches can only tighten.

4. What Is New

N1. A build gate that hashes only the comment-stripped numeric `static let` declarations of designated scoring-constant files, as a per-file sorted list of `path:symbol=value` lines under truncated SHA-256, and fails the build when that digest differs from a recorded value while the algorithm-version string still equals the version recorded beside it.

N2. Persisting the algorithm-version string on every stored score breakdown, decoding records written before the field existed to an explicit `unversioned` sentinel rather than to the current version, carrying the stamp through every derived copy, and rendering the stored stamp (or a "scored before versions were recorded" line) on the score-detail screen next to a fixed statement that the weights are calibrated, not outcome-validated.

N3. A containment gate that confines a hand-set ranking-weight identifier to a single source directory, so that a number with no outcome evidence cannot be displayed as a confidence anywhere else in the app without a visible rename.

N4. A machine-checked heuristic register in which the status vocabulary is defined by the register's own `_status_meanings` map, and in which each entry must carry non-empty affected-outputs, evidence and allowed-language fields and may cite only test files that exist.

N5. Requiring the register's declared scoring version to equal the code's current version, so that every algorithm-version change fails the build until the register is edited.

N6. A retracted-claims list kept inside the register whose strings fail the build if they reappear in any register entry or in any Swift or Markdown file under the app sources and docs, making the register the only place a corrected error may be written.

N7. Name-shape discovery of score-adjusting functions (names ending in Adjustment, Penalty, Bonus, Modifier, Damper or Fraction) in the analysis layer, where every discovered function must be claimed by a register entry's `symbols` list or waived by name with a reason, every claimed or waived name must still exist, and the register's entry count is a floor that only rises.

N8. A planted-violation harness that first confirms each gate is green, then applies a recorded mutation to a real file, requires a non-zero exit, restores the file, and finally refuses to report success while any `check_/enforce_` script lacks a plant, except those on a named build-only exemption list.

N9. A preflight gate that statically requires every gate's temp-file creation to go through a helper that exits non-zero on failure, with an `|| exit $?` guard at every command-substitution call site, and requires every sourced helper path to resolve, so gates fail closed rather than report clean after measuring nothing.

N10. Treating "goes red", "fails closed" and "is wired into the CI workflow" as three separately enforced properties of every quality gate, each with its own meta-gate, and each meta-gate itself covered by a planted violation.

N11. A curated production-mutation harness in which each mutation names a single test suite and an original text asserted to occur exactly once, the suite must be green beforehand, build failures are classified separately from test failures and retried once, and any skipped mutation fails the run.

N12. A static leak detector that parses the mutation harness's own source to recover each mutation's original text (including `chr()`-escaped forms) and fails if that text is absent from its target file or if any case cannot be parsed, combined with an atomic, PID-checked working-tree lock shared by both mutation harnesses.

N13. A budget ratchet over a directory of single-integer files in which the allowed direction is derived from the filename (`min_` prefix means a floor that may only rise, any other name a ceiling that may only fall), a loosening passes only when a commit in the compared range carries a `budget-raise-ok: <path>` trailer, and a CI-set flag turns a missing baseline into a failure.

N14. A documentation gate that rejects any Markdown line naming a ratcheting budget file next to a bare number, so prose must point at the value instead of restating it.

N15. Composing N1, N5, N7 and a lexical copy perimeter into one chain in which a change to a scoring constant forces a version change, the version change forces an edit of the evidence register that states each heuristic's permitted wording, and that wording is policed for prohibited vocabulary by the copy perimeter.

Not claimed (prior art).

- Recording a baseline of existing lint findings and failing only on new ones: SwiftLint's baseline option and betterer [6].
- Mutation testing and mutation scores in general [9][10][13][14].
- Version identifiers and documentation cards for models [7][8], and semantic versioning of software [15].
- Cryptographic hashing of configuration or lockfiles to detect change.

- Git commit trailers as a convention.
- Traceability between requirements, risk controls and verification as required by medical-device standards [11][12][16].
- The HRV methods named in the register, which are cited to their published sources [1][2].
- The general observation that ML and data systems accumulate configuration and documentation debt [17][18].

5. Accepted Tradeoffs and Limitations

5.1 Constants-hash lock

- **It compares against the recorded line only.** The gate cannot tell a version bump from a re-recording of the hash under the old version: rewriting the baseline file with the new hash and the same version passes. This happened on 2026-10-06 in commit 14ecd58, which replaced two training-readiness monotony-strain thresholds with one. The hash was re-recorded under v3.1.oct2026. Nothing in the tooling detected it. A planned remedy (not implemented) is a ratchet that refuses a changed hash under an unchanged version across commits.
- **A bumped version with a stale baseline disarms the gate.** The violation requires `version == recorded_version`. After the version string changes, the gate passes until someone re-records the baseline, and in that interval further constant changes also pass. Nothing forces the re-record.
- **Coverage is two files and numeric literals only.** The composite weights themselves (HRV 60 / sleep 25 / vitals 15, the two-input and comeback-mode splits, the sleep sub-score weights) live in a separate weights namespace in the general constants file. That file is not hashed: 30 numeric weight declarations sit outside the lock at this revision. Inline literals in scoring functions are not hashed either, for example the 90–100 verdict band. Neither is any constant whose right-hand side is an expression. The script's own header says a changed weight moves the hash; for the composite weights it does not.
- **Only `//` comments are stripped.** A `/* */` block comment containing a matching declaration would be hashed. Neither file has one today.
- **64-bit truncation.** 16 hex characters is ample for detecting accidental change. It is not intended to resist deliberate collision.
- **The ranking-weight check is a substring match** on one identifier. Copying the value into a differently named variable would evade it.
- **The version is shown in one place.** Only the score-detail screen renders a stored score's stamp. The assistant reports the current version, not the version that produced the score it is discussing.

5.2 Science register

The register states its own limits, quoted verbatim:

"CANNOT: prove scientific completeness. Discovery is name-shaped (functions ending Adjustment / Penalty / Bonus / Modifier / Damper / Fraction), so a heuristic expressed some other way is invisible to it, and no gate can judge whether a status is HONEST. This is governance over a known failure class, not proof that nothing was omitted."

and, from the same field: "'ten are awaiting validation' is machine-checked; 'every operative heuristic is classified' is not." (That sentence predates the current count; 12 entries are `awaiting-validation` at this revision.)

Further limitations found in the code:

- The status vocabulary is whatever `_status_meanings` contains, so adding a new key widens the vocabulary without any other check.
- `tests` is optional. One entry (`hrr-12bpm-band`, `supported-transfer`) cites no test. A cited test is checked for existence, not for whether it exercises the heuristic.
- The version-match rule forces an edit of one field, not a re-read of the entries.
- No gate compares user-facing copy with an entry's `allowed_language`.
- Discovery excludes the cause-detection directory, and it sees only `func` declarations. A heuristic written as a constant, a closure or an inline expression is invisible.
- The retracted-claims scan covers register entries, app sources and docs. It does not cover string catalogues, scripts or tests.

Validation status at this revision, in the register's words:

- `validated` (3): `hrv-time-domain-arithmetic`, `lnrmssd-swc-band`, `dfa-artifact-rejection`.
- `supported-transfer` (4): `dfa-a1-exercise-threshold`, `hrr-12bpm-band`, `training-load-ladder`, `tanaka-hrmax`.
- `awaiting-validation` (12): `resting-dfa-a1-reference-band`, `lfhf-window-filter`, `pns-sns-gap-readiness`, `acwr-readiness-damper`, `composite-weights-60-25-15`, `sleep-score-six-factor`, `hrv-sleep-staging`, `vitals-illness-pattern`, `spo2-flat-penalty`, `cause-ranking-weights`, `rmssd-absolute-fallback-bands`, `baseline-staleness-penalty`.
- `scheduled-for-removal`: none.

The recovery score as a whole is not validated against outcomes. Its composite weights are `awaiting-validation`.

5.3 Meta-gates

- **Plant detection is a filename grep.** The every-gate-has-a-plant check passes if the gate's filename appears anywhere in the harness source, including a comment or an echo line. It does not prove a working plant exists for that gate.
- **A plant tests one shape.** For example, the scoring-governance plant exercises the hash arm but not the ranking-weight arm. The science-register plants exercise the retracted-claim and unknown-status checks but not discovery, test-path existence or version match.
- **Skips do not fail the gate harness.** A gate that is already red, or a mutation that cannot be applied, prints SKIP and leaves the exit status at zero. The test-mutation harness, by contrast, fails on any skip.
- **The wiring check is also a substring test** against one workflow file. A filename mentioned in a comment counts as wired. The per-push Linux workflow is not audited.
- **The preflight gate covers one failure class:** unguarded temp-file creation and unresolvable source paths. It does not prove a gate fails closed on every missing input.
- **The test-mutation harness runs only by hand.** Its 125 cases were not run for this paper. Its standing evidence is the static leak check, which runs on every push.

5.4 Ratchets

- **Integer-only parsing.** A budget file whose value is not a plain integer is skipped. The views-coverage floor holds 43.5, so it is never compared. The documentation gate skips it for the same reason. The constants-hash file is likewise outside the ratchet.
- **The trailer's reason is not enforced.** The parser captures only the path token, so `budget-raise-ok: <path>` with no reason is accepted.
- **Deleting a budget file is not detected,** because the loop iterates over files present in the working tree.
- **Local runs skip by default.** With no baseline and no require flag, the gate exits 0, by design, so that uncommitted local work does not need a justification addressed to no one.
- **Measured ceilings tolerate existing debt.** The record calls them "debt inventories rather than assurance targets".

5.5 Scope

None of this verifies that the science is right. The governance establishes that numbers, versions, statuses, citations and wording stay consistent with one another and cannot change silently. Whether a status is honest is a human judgement, recorded in a file the build forces people to touch.

6. Comparison to Existing Approaches

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
SwiftLint baseline	New lint violations fail; recorded ones are tolerated	Not numeric; the baseline can be regenerated	No	No	No	SwiftLint documentation
betterer	A test's result may not get worse than the stored result; improvements are written back	Per-test "better" comparator	No	No	No	[6]
Model cards	Structured documentation of intended use, evaluation and caveats	Not applicable	Documents a model version; not build-enforced	Yes, as prose	No	[7]
ML model registries (e.g. MLflow)	Versioned model artifacts with stage transitions	Not applicable	Versions artifacts; does not watch source constants	Metadata, free-form	No	[8]

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
PIT / Stryker mutation testing	Mutation score over broad automatic mutations; Stryker can fail the build below a configured threshold	Threshold set by the user	No	No	Tests only, by percentage	[9][10]
IEC 62304 / ISO 13485	Documented life-cycle, change control, traceability, audited	Process, not numeric	Through configuration management, as a process	Through design and risk files, as a process	Through verification records, as a process	[11][12][16]
Emuqu (this paper)	Constants-hash/version lock; claim register with closed vocabulary, test-existence, retracted claims and discovery; per-file direction ratchets	Ceilings fall and <code>min_</code> floors rise, checked against a baseline commit	Yes, for numeric constants in two files	Yes, machine-checked fields, versioned with the code	Yes: planted violation per gate, preflight, wiring, 125 targeted test mutations	This paper

Emuqu is a consumer wellness app. It is not developed under IEC 62304 or ISO 13485 and claims no conformity with them. They appear in this table only because traceability and controlled change are what those standards address by process, and the mechanisms here address a narrow part of the same concern by build-time checks.

7. Provenance and Dates

- **Author and sole committer:** Chris Sharp chrissharp80@gmail.com.
- **Private development repository** `chrissharp80/emuqu-dev`: first commit 2026-08-16 (squashed from earlier work).
- **Public repository** `github.com/chrissharp80/emuqu`, PolyForm Strict 1.0.0: first commit 2026-09-08.
- **Earlier product name "Flow Recovery"**. The author's own records include app-generated "Flow_Recovery_.pdf" session reports in the author's Google Drive dated 2026-02-15, 2026-02-17 and 2026-03-03; a "Flow Recovery - Bluetooth" screen recording dated 2026-03-23; and a "flow-recovery-audit.md" dated 2026-05-10.

First appearance in emuqu-dev git history (per-file first-add dates):

Component	First added
Copy-perimeter linter; tech-debt budget counter	2026-08-16
Budget monotonicity ratchet	2026-08-22
Documentation-links gate; perimeter sync gate	2026-08-26

Component	First added
CI posture document; evidence report	2026-08-27
Planted-violation harness; preflight gate and helper library; ADR 004 (ratcheted budgets)	2026-08-28
Targeted test-mutation harness	2026-08-28
Wiring gate; leaked-mutation detector	2026-09-02
Scoring-version source; science register and its gate; scoring-governance gate and constants-hash baseline; per-push Linux gates workflow	2026-09-03

Public scoring-version history.

- v2.may2026: present at the public initial commit, 2026-09-08.
- v3.oct2026: 2026-10-03.
- v3.1.oct2026: 2026-10-04.
- Constants hash re-recorded under v3.1.oct2026: 2026-10-06.

This paper: published 2026-10-08. It describes source revision e028039 (2026-10-07).

References

1. Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology (1996). Heart rate variability: standards of measurement, physiological interpretation and clinical use. *Circulation* 93(5):1043–1065.
2. Plews DJ, Laursen PB, Stanley J, Kilding AE, Buchheit M (2013). Training adaptation and heart rate variability in elite endurance athletes: opening the door to effective monitoring. *Sports Medicine* 43(9):773–781.
3. Peake JM, Kerr G, Sullivan JP (2018). A critical review of consumer wearables, mobile applications, and equipment for providing biofeedback, monitoring stress, and sleep in physically active populations. *Frontiers in Physiology* 9:743.
4. Billman GE (2013). The LF/HF ratio does not accurately measure cardiac sympatho-vagal balance. *Frontiers in Physiology* 4:26.
5. Impellizzeri FM, Tenan MS, Kempton T, Novak A, Coutts AJ (2020). Acute:chronic workload ratio: conceptual issues and fundamental pitfalls. *International Journal of Sports Physiology and Performance* 15(6):907–913.
6. Phenomnomnominal (Craig Spence). betterer: incremental improvement test runner. Open-source project, github.com/phenomnomnominal/betterer.
7. Mitchell M, Wu S, Zaldivar A, Barnes P, Vasserman L, Hutchinson B, Spitzer E, Raji ID, Gebru T (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, 220–229.
8. Zaharia M, Chen A, Davidson A, et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41(4):39–45.

9. Coles H, Laurent T, Henard C, Papadakis M, Ventresque A (2016). PIT: a practical mutation testing tool for Java. *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016)*, tool demonstration, 449–452.
10. Stryker Mutator. Stryker documentation: mutation score and thresholds. stryker-mutator.io.
11. IEC 62304:2006+AMD1:2015. Medical device software — Software life cycle processes. International Electrotechnical Commission.
12. ISO 13485:2016. Medical devices — Quality management systems — Requirements for regulatory purposes. International Organization for Standardization.
13. Jia Y, Harman M (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37(5):649–678.
14. Petrović G, Ivanković M (2018). State of mutation testing at Google. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 163–171.
15. Preston-Werner T. Semantic Versioning 2.0.0. semver.org.
16. ISO 14971:2019. Medical devices — Application of risk management to medical devices. International Organization for Standardization.
17. Sculley D, Holt G, Golovin D, et al. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*.
18. Breck E, Cai S, Nielsen E, Salib M, Sculley D (2017). The ML test score: a rubric for ML production readiness and technical debt reduction. *IEEE International Conference on Big Data*, 1123–1132.

Appendix A: Parameter Table

Name	Value	Unit	Role
Hashed source files	2	files	Recovery-score constants file and HRV thresholds file
Hashed constants	155 (91 + 64)	declarations	Count at this revision
Constant regex	<code>static let\s+(\w+)\s*:\s*?\s*[\w<>\\[\],.]*=\s*([-\d.]+)</code>	—	Extracts symbol and literal value
Comment strip (hash)	<code>//[^\n]*</code> → empty	—	Line comments only
Hash line format	<code><path>:<symbol>=<value>\n</code>	UTF-8	One per constant, per-file sorted
Digest	SHA-256, first 16 hex chars	64 bits	Constants fingerprint
Baseline line	<code><version> <hash></code>	—	Recorded pair
Current version	v3.1.oct2026	—	Version this build computes
Legacy sentinel	<code>unversioned</code>	—	Decoded value for unstamped records
Ranking-key identifier	<code>rankingWeight</code>	—	Allowed only in the cause-detection directory
Register entries	19	entries	Count at this revision

Name	Value	Unit	Role
Status vocabulary	validated, supported-transfer, awaiting-validation, scheduled-for-removal	—	Keys of <code>_status_meanings</code>
Required non-empty fields	affects, evidence, allowed_language	—	Per entry
Retracted strings	3	strings	Scanned in entries, app sources and docs (.swift, .md)
Discovery suffixes	Adjustment, Penalty, Bonus, Modifier, Damper, Fraction	—	Function-name shape
Discovery exclusions	names ending Descriptions or Message; cause-detection directory	—	Not score inputs
Discovered functions	13 (12 claimed, 1 waived)	functions	At this revision
Gate scripts	45	scripts	<code>check_*</code> and <code>enforce_*</code>
Planted violations	64	plants	Planted-violation harness
Plant exemptions	coverage-floor gate, Thread Sanitizer gate	—	Need a build or simulator
Targeted test mutations	125	mutations	Test-mutation harness
Build-failure retry	1	retry	Baseline suite run only
Wiring exemptions	1	gate	Coverage gate runs inside the coverage step
Preflight-scanned scripts	53	scripts	All shell scripts except the preflight gate
Gate exit codes	0 clean, 1 violation, 2 configuration error	—	Scoring, register, wiring gates
Helper exit code	70 (EX_OSERR)	—	Temp-file or empty-input failure
Lock-held exit code	75 (EX_TEMPFAIL)	—	Another mutator is live
Trailer	<code>budget-raise-ok: <path> <reason></code>	—	Acknowledged loosening; case-insensitive key
Floor prefix	<code>min_</code>	—	Budget may only rise
Require-baseline flag	<code>BUDGET_REQUIRE_BASELINE</code>	env var	Missing baseline becomes a failure
Doc-quote number regex	<code>(?![\w.]) (\d{2,6}) (?![\w.%])</code>	—	Bare number near a budget filename
Doc-quote window	current line + next line	lines	Where the number is sought
macOS billing multiplier	10	× Linux	CI cost basis
Full macOS run	~108 wall / ~1,080 billable	minutes	Measured by the repository

Appendix B: Pseudocode

B.1 Constants-hash version lock

```
function scoring_governance():
    problems = []
    for file in all_swift_files(APP_SOURCES):
        if file not under CAUSE_DETECTION_DIR:
            for (n, line) in lines(file):
                if "rankingWeight" in line: problems.add(file, n)
    h = sha256()
    count = 0
    for path in [RECOVERY_CONSTANTS, HRV_THRESHOLDS]:           # fixed order
        if not exists(path): exit 2
        src = regex_replace(read(path), "//[^\n]*", "")
        pairs = sorted(regex_findall(src, CONST_RE))             # (symbol, value)
        for (sym, val) in pairs:
            h.update(utf8(path + ":" + sym + "=" + val + "\n"))
            count += 1
    current_hash = hex(h)[0:16]
    version = regex_group(read(VERSION_FILE), 'static let current\s*=\s*"([^\n]+)"') or
exit 2
    if not exists(BASELINE): exit 2
    (rec_version, rec_hash) = split_whitespace(read(BASELINE))
    if current_hash != rec_hash and version == rec_version:
        problems.add("constants changed without a version change")
    print(count, current_hash, version)
    exit 1 if problems else 0

decode ScoreBreakdown(record):
    scoringVersion = record.get("scoringVersion") ?? "unversioned"
```

B.2 Science register gate

```

function science_register():
    reg = parse_json(REGISTER) or exit 2
    VALID = keys(reg._status_meanings); if empty: exit 2
    problems = []; seen = {}
    for e in reg.entries:
        if e.id in seen: problems.add(dup)
        seen.add(e.id)
        if e.status not in VALID: problems.add(status)
        for f in [affects, evidence, allowed_language]: if empty(e[f]): problems.add(f)
        for t in e.tests or []: if not exists(t): problems.add(t)
    if reg.scoring_version != read_current_version(): problems.add(version)
    for c in reg._retracted_claims.claims:
        for e in reg.entries, f in [evidence, affects, allowed_language, notes]:
            if c in join(e[f]): problems.add(c)
        for file in files(APP_SOURCES, DOCS) with ext in {swift, md}:
            if c in read(file): problems.add(file, c)
    discovered = {}
    for file in swift_files(ANALYSIS_DIR) not under CAUSE_DETECTION_DIR:
        for name in findall("func\s+([a-zA-Z]+
(?:Adjustment|Penalty|Bonus|Modifier|Damper|Fraction))\b"):
            if not name.endswith("Descriptions" or "Message"): discovered.add(name)
    claimed = union(e.symbols for e in entries)
    waived = {k in reg._waived_symbols if not k.startswith("_")}
    for n in discovered - claimed - waived: problems.add(unclassified n)
    for n in (claimed u waived) - discovered: problems.add(stale n)
    print tally
    if problems: exit 1
    if len(entries) < read_int(REGISTER_FLOOR): exit 1
    exit 0

```

B.3 Planted-violation harness

```

for (name, gate, file, mutation) in PLANTS:
    if run(gate) != 0: SKIP; continue
    backup(file)
    if not run_python(mutation): restore(); SKIP; continue
    code = run(gate); restore_all()
    if code != 0: PASS else: FAIL
    unplanted = [g for g in glob(check_*.sh, enforce_*.sh)
                  if g != "check_gate_preflight.sh" and g not in NEEDS_BUILD
                  and basename(g) not in read(this_script)]
    if unplanted: exit 1
    exit 1 if FAIL > 0 else 0

```

B.4 Preflight and wiring

```

preflight:
  for s in shell_scripts except self:
    for line in non_comment_lines(s):
      if matches("(?!gate_)\bmkttemp\b") and "gate_mktemp" not in line: fail
      if "gate_mktemp" in line and "|| exit" not in line: fail
      if line sources "$(dirname "$0")/X" and not exists(scripts/X): fail

wiring:
  wf = read(WORKFLOW) or exit 2
  for g in gates (check_*/enforce_*) not in NOT_A_STEP:
    if basename(g) not substring of wf: unwired.add(g)
  exit 1 if unwired else 0

```

B.5 Targeted test mutations and leak detector

```

run_suite(s): xcodebuild test -only-testing s
  -> 0 passed | 2 if build banner or no "Test Suite|Test Case" | 1 otherwise
for (name, suite, file, mutation) in MUTATIONS:
  b = run_suite(suite); if b == 2: b = run_suite(suite)
  if b != 0: SKIP; continue
  backup(file); if not apply(mutation): restore(); SKIP; continue
  r = run_suite(suite); restore()
  if r != 0: PASS else: FAIL
exit 1 if FAIL or SKIP else 0

leak_detector:
  for case in split(read(MUTATION_SCRIPT), '\nmutate ')[1:]:
    path = first double-quoted app-directory .swift path in case
    orig = group(o='(.*?)';assert) or group(s.count('(.*?)')\s*[=><])
    if not path or not orig: unparsed.add; continue
    orig = unescape(orig); orig = resolve("'" + chr(N) + '" -> char N)
    if "chr(" in orig or "'" + " in orig: unparsed.add; continue
    if not exists(path) or orig not in read(path): leaked.add
  exit 1 if leaked or unparsed else 0

tree_lock_acquire(owner):
  if mkdir(LOCK) fails:
    if pid_alive(read(LOCK/pid)): exit 75
    warn "stale lock: previous run never restored"; rm LOCK; mkdir(LOCK) or exit 70
  write LOCK/owner, LOCK/pid

```

B.6 Direction-aware ratchet

```

base = arg or ("origin/"+GITHUB_BASE_REF) or BUDGET_BASE_REF
if not base or base == 40 zeros or not resolvable(base):
    if BUDGET_REQUIRE_BASELINE: exit 1 else: exit 0
acks = [path for msg in commit_messages(base..HEAD)
        for path in regex_i("^budget-raise-ok:\s*(\S+)", msg)]
for f in sorted(CI_DIR/*.txt):
    cur = int_or_skip(read(f)); prev = int_or_skip(git_show(base, f)) or skip
    if cur == prev: continue
    loosened = (cur < prev) if basename(f).startswith("min_") else (cur > prev)
    if loosened and f not in acks: violations.add(f, prev, cur)
exit 1 if violations else 0

count_for_budget(src):
    src = blank_block_comments_keep_newlines(src)
    src = blank_line_comments_same_length(src)
    src = blank(PLATFORM_SINGLETON.shared)
    return count(pattern, src)

```

Appendix C: Source Map

Method	Repository paths
Constants-hash lock, ranking- weight containment	scripts/check_scoring_governance.sh, .ci/scoring_constants_hash.txt (baseline), Emuqu/Sources/Utilities/Constants+RecoveryScore.swift, Emuqu/Sources/Analysis/HRVThresholds.swift
Version string and sentinel	Emuqu/Sources/Analysis/ScoringVersion.swift
Version persisted on breakdowns	Emuqu/Sources/Analysis/RecoveryScoreCalculator.swift, Emuqu/Sources/Analysis/VitalsScoring.swift, Emuqu/Sources/Storage/CloudKitSyncSupport.swift
Version shown	Emuqu/Sources/Views/RecoveryScoreDetailView+Sections.swift
Version given to the assistant	Emuqu/Sources/Assistant/Facts/AppFactResolver.swift, Emuqu/Sources/Assistant/Context/ContextBuilder.swift
Composite weights (not hashed)	Emuqu/Sources/Utilities/Constants.swift
Science register and gate	Tools/science_register/register.json, scripts/check_science_register.sh, register floor file in .ci/
Copy perimeter (paper 03)	Tools/copy_linter/lint.py, Tools/copy_linter/prohibited_terms.json, scripts/check_perimeter_sync.sh
Planted violations	scripts/verify_gates_fail.sh
Fail-closed helpers and tree lock	scripts/lib/preflight.sh, scripts/check_gate_preflight.sh
Wiring	scripts/check_gates_wired.sh, .github/workflows/ci.yml

Method	Repository paths
Targeted test mutations and leak backstop	<code>scripts/verify_tests_fail.sh</code> , <code>scripts/check_no_leaked_mutations.sh</code>
Ratchets	<code>docs/adr/004-ratcheted-budgets.md</code> , <code>scripts/check_budget_monotonicity.sh</code> , <code>scripts/enforce_tech_debt_budgets.sh</code> , <code>scripts/enforce_swifflint_budget.sh</code> , budget files in <code>.ci/</code>
Documentation constants rule	<code>scripts/check_doc_links.sh</code>
CI posture and evidence	<code>docs/CI_POSTURE.md</code> , <code>.github/workflows/gates.yml</code> , <code>.github/workflows/ci.yml</code> , <code>scripts/evidence_report.sh</code> , <code>Makefile</code>